Group No: 04

Group Members

SL	Name	Student ID (Roll)
1	Md. Masud Rana	2211176104
2	Md. Shibli Noman Sarkar	2210776130
3	Md. Majnu Miah	2210776138
4	Md. Ruhul Amin	2210776147
5	Md. Naimur Rahman	2111077144

```
Start coding or generate with AI.
```


```
Start coding or generate with AI.
```


CNN-Based MNIST Image Classification

1. Overview

This project uses a **Convolutional Neural Network (CNN)** to classify handwritten digit images from the **MNIST dataset**.

The implementation is developed using **Python** and the **TensorFlow/Keras** deep learning framework. The project uses convolutional layers, pooling layers, fully connected layers, and dropout to build an image classification model.

The following Python libraries and modules are imported:

```
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
from tensorflow.keras.callbacks import EarlyStopping
```

```
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, F
from tensorflow.keras.callbacks import EarlyStopping
```

Loading the MNIST Dataset

The following Python code is used to load the **MNIST handwritten digit dataset**:

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

1. Purpose

The `mnist.load_data()` function loads the **MNIST dataset**, which is a widely used dataset for image classification and machine learning experiments.

The dataset contains images of handwritten digits from **0 to 9**.

Each image is a grayscale image with a size of **28 × 28 pixels**.

---

2. Code Explanation

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

The `mnist.load_data()` function returns four datasets:

- `X_train` → Training images
- `y_train` → Training labels
- `X_test` → Testing images
- `y_test` → Testing labels

These datasets are divided into two main groups:

1. **Training Dataset**
2. **Testing Dataset**

---

3. Training Dataset

`X_train`

`X_train` contains the images that are used to **train the neural network**.

Each image has dimensions:

```
28 × 28 pixels
```

The standard MNIST dataset contains:

```
60,000 training images
```

Therefore, the shape of `X_train` is:

```
X_train.shape
```

Output:

```
(60000, 28, 28)
```

This means:

- `60000` → Number of training images
- `28` → Image height
- `28` → Image width

Since MNIST images are grayscale, each pixel contains an intensity value between **0 and 255**.

`y_train`

`y_train` contains the corresponding **labels** for the training images.

Each label represents a digit from:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

There are 60,000 training labels.

The shape is:

```
y_train.shape
```

Output:

```
(60000,)
```

For example:

```
y_train[0] = 5
```

This means the first image in `X_train` represents the handwritten digit **5**.

---

4. Testing Dataset

`X_test`

`X_test` contains images that are used to **evaluate the trained model**.

The standard MNIST dataset contains:

```
10,000 testing images
```

The shape of `X_test` is:

```
X_test.shape
```

Output:

```
(10000, 28, 28)
```

This means there are 10,000 images, and each image is 28 × 28 pixels.

`y_test`

`y_test` contains the correct labels corresponding to the testing images.

The shape is:

```
y_test.shape
```

Output:

```
(10000,)
```

For example:

```
y_test[0] = 7
```

This indicates that the first test image represents the digit **7**.

---

5. Dataset Structure

The MNIST dataset can be represented as follows:

```
MNIST Dataset
│
├── Training Dataset
│   ├── X_train → 60,000 images
│   └── y_train → 60,000 labels
│
└── Testing Dataset
    ├── X_test → 10,000 images
    └── y_test → 10,000 labels
```

---

6. Relationship Between Images and Labels

Each image has a corresponding label.

For example:

```
X_train[0]  → Image of digit 5
y_train[0]  → 5
```

Similarly:

```
X_test[0]   → Image of digit 7
y_test[0]   → 7
```

The image and its label have the same index.

Therefore:

```
X_train[i]
```

is associated with:

```
y_train[i]
```

---

7. Why MNIST Is Used

MNIST is commonly used for learning and testing machine learning and deep learning algorithms because:

- It is relatively small and easy to process.
- The images have a fixed size of 28 × 28 pixels.
- It contains 10 different classes.
- It is suitable for classification problems.
- It is widely used for testing neural networks and CNN models.

---

8. Use in CNN Models

The MNIST dataset is particularly suitable for **Convolutional Neural Networks (CNNs)**.

Before feeding the images into a CNN, the image data is often reshaped to include a **channel dimension**:

```
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

The final shape becomes:

```
X_train → (60000, 28, 28, 1)
X_test  → (10000, 28, 28, 1)
```

Here, `1` represents the single grayscale channel.

---

9. Summary

The following code:

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

loads the MNIST handwritten digit dataset and separates it into training and testing data.

Variable	Description	Shape
`X_train`	Training images	`(60000, 28, 28)`
`y_train`	Training labels	`(60000,)`
`X_test`	Testing images	`(10000, 28, 28)`
`y_test`	Testing labels	`(10000,)`

The **training data** is used to teach the CNN how to recognize handwritten digits, while the **testing data** is used to measure how accurately the trained model can classify unseen images.

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Displaying the First Five Training Images

After loading the MNIST dataset, it is useful to visualize some training images to understand the type of data being used.

The following code displays the **first five images from the training dataset** along with their corresponding labels.

Code

```
# Display first 5 training images

plt.figure(figsize=(10, 2))

for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')

plt.show()
```

> **Note:** Proper indentation is required inside the `for` loop. Also, `X_train[i]` should be written without the backslash shown in the original code.

---

1. Purpose of the Code

The main purpose of this code is to **visually inspect the first five training images** of the MNIST dataset.

Visualization helps us:

- Verify that the dataset was loaded correctly.
- Understand the appearance of the input images.
- Confirm that the images contain handwritten digits.
- Check whether the labels correspond correctly to the images.
- Gain a basic understanding of the dataset before training the CNN model.

---

2. Creating the Figure

```
plt.figure(figsize=(10, 2))
```

The `plt.figure()` function creates a new figure for displaying the images.

The `figsize` parameter defines the size of the figure:

```
figsize = (width, height)
```

In this case:

- Width = `10` inches
- Height = `2` inches

Therefore, a wide figure is created to display five images horizontally.

---

3. Iterating Through the Images

```
for i in range(5):
```

The `for` loop is used to process the first five training images.

The `range(5)` function generates:

```
0, 1, 2, 3, 4
```

Therefore, the loop accesses:

```
X_train[0]
X_train[1]
X_train[2]
X_train[3]
X_train[4]
```

These represent the first five images in the training dataset.

---

4. Creating Subplots

```
plt.subplot(1, 5, i + 1)
```

The `subplot()` function divides the figure into multiple smaller sections.

The syntax is:

```
plt.subplot(rows, columns, position)
```

Here:

```
rows    = 1
columns = 5
position = i + 1
```

Therefore, the figure is divided into **1 row and 5 columns**.

The five images are displayed like this:

```
┌─────────┬─────────┬─────────┬─────────┬─────────┐
│ Image 1 │ Image 2 │ Image 3 │ Image 4 │ Image 5 │
└─────────┴─────────┴─────────┴─────────┴─────────┘
```

---

5. Displaying the Image

```
plt.imshow(X_train[i], cmap='gray')
```

The `imshow()` function displays an image.

`X_train[i]`

This selects the `i`-th image from the training dataset.

For example:

```
X_train[0]
```

selects the first training image.

`cmap='gray'`

The `cmap` parameter specifies the color map used to display the image.

Since MNIST images are **grayscale images**, the `gray` color map is used.

The pixel values represent different shades of gray:

```
0   → Black
255 → White
```

---

6. Displaying the Label

```
plt.title(f"Label: {y_train[i]}")
```

This adds the corresponding label above each image.

The label is obtained from:

```
y_train[i]
```

For example, if:

```
y_train[0] = 5
```

the title will appear as:

```
Label: 5
```

This allows us to visually compare the image with its actual class.

---

7. Removing the Axis

```
plt.axis('off')
```

This removes the x-axis and y-axis from each image.

Without this command, the image would be displayed with axis values around it.

Since the axes are not important for identifying handwritten digits, they are hidden to make the visualization cleaner.

---

8. Displaying the Figure

```
plt.show()
```

The `plt.show()` function displays the complete figure containing all five images.

The final output will contain five handwritten digit images arranged horizontally, with each image showing its corresponding label.

---

9. Expected Output

The output will look conceptually like this:

```
      Label: 5   Label: 0   Label: 4   Label: 1   Label: 9

        5          0          4          1          9
      [image]    [image]    [image]    [image]    [image]
```

The exact digits depend on the order of the MNIST training dataset.

---

10. Summary

This visualization code provides a simple way to inspect the MNIST training data before building and training the CNN model.

Code	Purpose
`plt.figure()`	Creates the figure
`for i in range(5)`	Iterates over the first five images
`plt.subplot()`	Creates five image positions
`plt.imshow()`	Displays each image
`cmap='gray'`	Displays the image in grayscale
`plt.title()`	Displays the image's label
`plt.axis('off')`	Hides the axes
`plt.show()`	Displays the final visualization

Overall Process

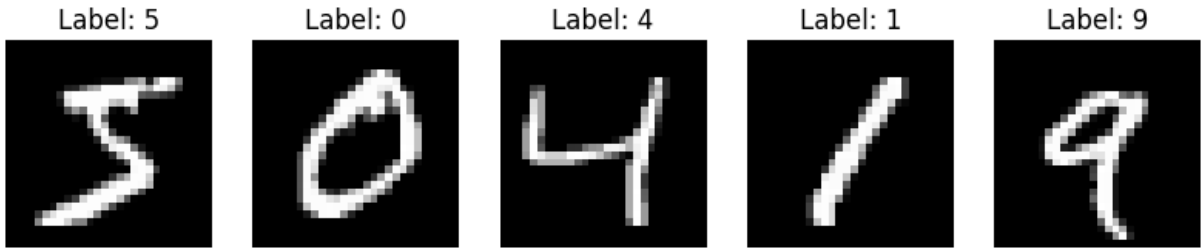
```
MNIST Dataset
      ↓
Select first 5 training images
      ↓
Create 1 × 5 subplot
      ↓
Display images in grayscale
      ↓
Add corresponding labels
      ↓
Hide axes
      ↓
Display final figure
```

This step is an important part of **exploratory data analysis (EDA)** because it confirms that the input images and their labels are correctly loaded before proceeding to preprocessing and CNN model training.

```
# Display first 5 training images
plt.figure(figsize=(10, 2))

for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')

plt.show()
```



Normalization of Image Data

Before training the CNN model, the MNIST image data is **normalized** so that the pixel values are converted from the original range of **0–255** to a smaller range of **0–1**.

The following code performs this normalization:

```
# Normalization

X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
```

---

1. Purpose of Normalization

MNIST images contain grayscale pixel values ranging from:

```
0 to 255
```

For example:

- `0` represents black.
- `255` represents white.
- Values between `0` and `255` represent different shades of gray.

Neural networks generally perform better and train more efficiently when input values are kept within a smaller numerical range.

Therefore, the pixel values are normalized to:

```
0 to 1
```

---

2. Converting the Data Type

```
X_train = X_train.astype("float32")
```

The `astype()` function changes the data type of the training images to **32-bit floating-point numbers**.

The original MNIST image data is commonly stored as an integer type such as `uint8`.

For example:

```
Original:
0, 128, 255

After conversion:
0.0, 128.0, 255.0
```

Using `float32` is useful because neural network calculations typically use floating-point values.

---

3. Dividing by 255

```
X_train = X_train.astype("float32") / 255.0
```

Each pixel value is divided by `255`.

Since `255` is the maximum possible pixel intensity in an 8-bit grayscale image, this converts the values from:

```
0 – 255
```

to:

```
0 – 1
```

For example:

Original Pixel Value	Normalized Value
0	0.000
64	0.251

Original Pixel Value	Normalized Value
128	0.502
192	0.753
255	1.000

The mathematical operation is:

```
Normalized Pixel = Original Pixel / 255
```

For example, if a pixel has a value of `128`:

```
128 / 255 ≈ 0.502
```

Therefore, the normalized pixel value becomes approximately `0.502`.

---

4. Normalizing the Training Data

```
X_train = X_train.astype("float32") / 255.0
```

This operation is applied to **every pixel of every training image**.

The training data changes from:

```
0 ≤ X_train ≤ 255
```

to:

```
0 ≤ X_train ≤ 1
```

The number and dimensions of the images do not change.

For example:

```
Before normalization:
X_train.shape = (60000, 28, 28)

After normalization:
X_train.shape = (60000, 28, 28)
```

Only the **pixel values and data type** are changed.

---

5. Normalizing the Testing Data

```
X_test = X_test.astype("float32") / 255.0
```

The same normalization process is applied to the testing dataset.

This ensures that the training and testing data have the **same scale**.

The testing pixel values are also converted from:

```
0 – 255
```

to:

```
0 – 1
```

---

6. Why Both Training and Testing Data Are Normalized

It is important to normalize both datasets using the same transformation.

Training Data

```
X_train = X_train.astype("float32") / 255.0
```

is used when training the model.

Testing Data

```
X_test = X_test.astype("float32") / 255.0
```

is used when evaluating the trained model.

If the training data were normalized but the testing data were not, the model would receive data on two different scales. This could negatively affect evaluation performance.

---

7. Benefits of Normalization

Normalization provides several advantages for CNN training:

Faster Training

Smaller input values can help the optimization algorithm converge more efficiently.

Numerical Stability

Keeping input values in a small range can make neural network calculations more stable.

Better Gradient Behavior

Normalized inputs can help produce more manageable gradients during backpropagation.

Consistent Data Scale

Both training and testing images use the same numerical range.

---

8. Before and After Normalization

Before Normalization

```
Pixel values:
0, 1, 2, ..., 254, 255
```

Data type:

```
uint8
```

After Normalization

```
Pixel values:
0.000, 0.004, 0.008, ..., 0.996, 1.000
```

Data type:

```
float32
```

---

9. Complete Normalization Code

```
# Normalization
```

```
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
```

After executing this code, the image data is ready for the next preprocessing stage and can be provided as input to the CNN model.

---

10. Summary

Operation	Purpose
`astype("float32")`	Converts image values to 32-bit floating-point numbers
`/ 255.0`	Scales pixel values from 0–255 to 0–1
`X_train` normalization	Normalizes training images
`X_test` normalization	Normalizes testing images

Overall Process

```
Original MNIST Image
        ↓
Pixel values: 0 – 255
        ↓
Convert to float32
        ↓
Divide every pixel by 255
        ↓
Normalized Image
        ↓
Pixel values: 0 – 1
        ↓
Ready for CNN Training
```

Normalization is therefore an important **data preprocessing step** that prepares the MNIST images for efficient and stable CNN training.

```
1 # Normalization
2 X_train = X_train.astype("float32") / 255.0
3 X_test = X_test.astype("float32") / 255.0
```

Reshaping the Data for CNN

A Convolutional Neural Network (CNN) expects image data to have a specific structure. Therefore, after normalization, the MNIST images are reshaped to include a **channel dimension**.

The following code performs the reshaping:

```
# Reshape for CNN

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

---

1. Purpose of Reshaping

Originally, each MNIST image has the shape:

```
28 × 28
```

Therefore, the training dataset has the shape:

```
(60000, 28, 28)
```

However, a CNN expects image data in the following format:

```
(number of images, height, width, channels)
```

For MNIST, this becomes:

```
(60000, 28, 28, 1)
```

The final `1` represents the **single grayscale channel** of the image.

---

2. Reshaping the Training Data

```
X_train = X_train.reshape(-1, 28, 28, 1)
```

This changes the shape of `X_train`.

Before Reshaping

```
(60000, 28, 28)
```

The dimensions represent:

- `60000` → Number of training images
- `28` → Image height
- `28` → Image width

There is no explicit channel dimension.

After Reshaping

```
(60000, 28, 28, 1)
```

The dimensions now represent:

- `60000` → Number of images
- `28` → Height
- `28` → Width
- `1` → Number of channels

---

3. Meaning of `-1`

The `-1` in:

```
reshape(-1, 28, 28, 1)
```

tells NumPy to **automatically determine the number of images** based on the total number of elements.

For the MNIST training dataset, NumPy determines:

```
-1 = 60000
```

Therefore:

```
(-1, 28, 28, 1)
```

becomes:

```
(60000, 28, 28, 1)
```

This is useful because we do not need to manually specify the number of images.

---

4. Meaning of `28, 28`

The values:

```
28, 28
```

represent the spatial dimensions of each MNIST image.

```
28 → Height
28 → Width
```

Therefore, every image remains:

```
28 × 28 pixels
```

Reshaping does **not change the image size** or pixel values. It only changes how the data is organized in memory.

---

5. Meaning of `1`

The final value:

```
1
```

represents the number of **image channels**.

MNIST images are grayscale, so they contain only one channel:

```
Grayscale image → 1 channel
```

For comparison:

Image Type	Number of Channels
Grayscale	1
RGB	3
RGBA	4

Therefore, an MNIST image is represented as:

```
28 × 28 × 1
```

---

6. Reshaping the Testing Data

The same operation is applied to the test dataset:

```
X_test = X_test.reshape(-1, 28, 28, 1)
```

Before reshaping:

```
(10000, 28, 28)
```

After reshaping:

```
(10000, 28, 28, 1)
```

Here:

- `10000` → Number of test images
- `28` → Image height
- `28` → Image width
- `1` → Grayscale channel

---

7. Why CNN Requires This Format

CNN layers such as `Conv2D` process image data using spatial dimensions and channels.

A typical `Conv2D` input has the structure:

```
Batch Size × Height × Width × Channels
```

For the MNIST dataset:

```
60000 × 28 × 28 × 1
```

This allows the CNN to understand:

- The height of the image.
- The width of the image.
- The grayscale channel.
- The collection of images being processed.

For example, a CNN layer may be defined as:

```
Conv2D(
    32,
    (3, 3),
    activation='relu',
    input_shape=(28, 28, 1)
)
```

The `input_shape=(28, 28, 1)` tells the CNN that each input image is 28 × 28 pixels with one grayscale channel.

---

8. Reshaping Does Not Change the Data

It is important to understand that `reshape()` does not modify the actual pixel values.

For example, suppose an image originally has:

```
28 × 28
```

After reshaping, it becomes:

```
28 × 28 × 1
```

The pixel information remains exactly the same.

The operation simply changes the structure from:

```
Image
  ↓
28 × 28
```

to:

```
Image
  ↓
28 × 28 × 1
  ↓
Height × Width × Channel
```

---

9. Complete Reshaping Code

```
# Reshape for CNN

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

You can verify the new shapes using:

```
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

Expected output:

```
Training data shape: (60000, 28, 28, 1)
Testing data shape: (10000, 28, 28, 1)
```

---

10. Overall Preprocessing Pipeline

The MNIST data preprocessing performed so far can be summarized as:

```
MNIST Dataset
      ↓
Load Training & Testing Data
      ↓
Visualize Images
      ↓
Normalize Pixel Values
      ↓
Pixel Range: 0–255 → 0–1
```

```
      ↓
Reshape for CNN
      ↓
(60000, 28, 28) → (60000, 28, 28, 1)
(10000, 28, 28) → (10000, 28, 28, 1)
      ↓
Ready for CNN Model
```

---

11. Summary

Operation	Before	After
Training data	`(60000, 28, 28)`	`(60000, 28, 28, 1)`
Testing data	`(10000, 28, 28)`	`(10000, 28, 28, 1)`
Image size	`28 × 28`	`28 × 28`
Channels	Not explicitly included	`1` grayscale channel
Pixel range	`0–1` after normalization	`0–1`

Conclusion

The reshaping step prepares the MNIST images for the CNN by adding an explicit **channel dimension**. Each image is represented as **28 × 28 × 1**, which is the appropriate input format for a `Conv2D` layer when using TensorFlow/Keras with the default channels-last format.

```
1 # Reshape for CNN
2 X_train = X_train.reshape(-1, 28, 28, 1)
3 X_test = X_test.reshape(-1, 28, 28, 1)
```

Building the Convolutional Neural Network (CNN)

After preprocessing and reshaping the MNIST dataset, the next step is to build a **Convolutional Neural Network (CNN)** for handwritten digit classification.

The CNN is created using the Keras `Sequential` API.

Code

```
# Build CNN Model
```

```
model = Sequential([
    Conv2D(32, 3, activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),

    Conv2D(64, 3, activation='relu'),
    MaxPooling2D((2, 2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dropout(0.5),

    Dense(10, activation='softmax')
])
```

---

1. Purpose of the CNN

The purpose of this CNN is to classify handwritten digits from the MNIST dataset.

The model receives an image of size:

```
28 × 28 × 1
```

and predicts which digit the image represents:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Therefore, the CNN has **10 output classes**.

---

2. Sequential Model

```
model = Sequential([
    ...
])
```

`Sequential` is a Keras model type that allows layers to be added one after another in a linear sequence.

The data flows through the network as follows:

```
Input Image
    ↓
Conv2D (32 filters)
    ↓
MaxPooling
    ↓
Conv2D (64 filters)
    ↓
MaxPooling
    ↓
Flatten
    ↓
Dense (128 neurons)
    ↓
Dropout
    ↓
Dense (10 neurons)
    ↓
Output Prediction
```

---

3. First Convolutional Layer

```
Conv2D(32, 3, activation='relu', input_shape=(28, 28, 1))
```

The first `Conv2D` layer extracts important visual features from the input image.

Parameters

Parameter	Value	Description
Number of filters	`32`	Learns 32 different feature patterns
Kernel size	`3`	Uses a 3 × 3 convolution window
Activation	`relu`	Introduces non-linearity
Input shape	`(28, 28, 1)`	28 × 28 grayscale image

Input Shape

The model expects each image to have:

```
28 × 28 × 1
```

where:

- `28` = height
- `28` = width
- `1` = grayscale channel

32 Filters

The layer learns **32 different filters**.

These filters can learn features such as:

- Edges
- Lines
- Curves
- Corners
- Simple digit patterns

---

4. ReLU Activation

The convolutional layer uses:

```
activation='relu'
```

ReLU stands for **Rectified Linear Unit**.

It converts negative values to zero while keeping positive values.

Conceptually:

```
Negative value → 0
Positive value → unchanged
```

ReLU helps the neural network learn complex, non-linear patterns.

---

5. First Max Pooling Layer

```
MaxPooling2D((2, 2))
```

Max pooling reduces the spatial size of the feature maps.

A `2 × 2` pooling window is used.

The pooling operation selects the **maximum value** from each 2 × 2 region.

Main Benefits

- Reduces the size of feature maps.
- Reduces computational requirements.
- Helps retain important features.
- Provides some tolerance to small image shifts.

The spatial dimensions are approximately reduced by half.

---

6. Second Convolutional Layer

```
Conv2D(64, 3, activation='relu')
```

The second convolutional layer uses **64 filters**.

Compared with the first convolutional layer, it can learn more complex features.

The first layer may detect simple patterns such as:

```
Edges → Lines → Curves
```

The second layer can combine these features into more meaningful patterns such as:

```
Lines + Curves → Digit Components
```

Parameters

Parameter	Value
Number of filters	`64`
Kernel size	`3 × 3`
Activation	`ReLU`

Increasing the number of filters from **32 to 64** allows the model to learn a larger variety of features.

---

7. Second Max Pooling Layer

```
MaxPooling2D((2, 2))
```

The second max pooling layer again reduces the spatial dimensions of the feature maps.

This makes the representation smaller while retaining the most important detected features.

The combination of:

```
Convolution → ReLU → Pooling
```

is a common pattern in CNN architectures.

---

8. Flatten Layer

```
Flatten()
```

The `Flatten` layer converts the multi-dimensional feature maps into a **one-dimensional vector**.

Before flattening, the data has multiple dimensions:

```
Height × Width × Channels
```

After flattening:

```
One-dimensional vector
```

This is necessary because the next layer, `Dense`, expects a one-dimensional input.

For this architecture, the feature maps after the second pooling layer are:

```
5 × 5 × 64
```

Therefore, the flattened vector contains:

```
5 × 5 × 64 = 1600
```

features.

So the output of `Flatten()` has:

```
1600 values
```

---

9. Fully Connected Dense Layer

```
Dense(128, activation='relu')
```

The first fully connected layer contains **128 neurons**.

This layer receives the features extracted by the convolutional layers and learns how to combine them for classification.

The ReLU activation function is used again:

```
activation='relu'
```

The layer therefore produces a higher-level representation of the input image.

---

10. Dropout Layer

```
Dropout(0.5)
```

Dropout is a **regularization technique** used to reduce overfitting.

The value:

```
0.5
```

means that approximately **50% of the neurons in this layer are randomly ignored during each training update**.

For example:

```
128 neurons
     ↓
Randomly deactivate approximately 50%
     ↓
Remaining neurons continue learning
```

Dropout encourages the network to learn more robust features instead of depending too heavily on specific neurons.

> Dropout is active during training but is automatically disabled during normal evaluation and prediction.

---

11. Output Layer

```
Dense(10, activation='softmax')
```

The final layer contains **10 neurons**, corresponding to the ten possible MNIST classes:

```
0  1  2  3  4  5  6  7  8  9
```

The `softmax` activation converts the outputs into probability-like values.

For example, the model might produce:

```
Digit 0 → 0.01
Digit 1 → 0.02
Digit 2 → 0.03
Digit 3 → 0.01
Digit 4 → 0.01
Digit 5 → 0.87
Digit 6 → 0.01
Digit 7 → 0.01
Digit 8 → 0.01
Digit 9 → 0.02
```

The highest probability is:

```
Digit 5 → 0.87
```

Therefore, the model predicts the image as **digit 5**.

---

12. Complete CNN Architecture

The complete architecture can be summarized as:

```
Input
28 × 28 × 1
     ↓
Conv2D
32 filters, 3 × 3
ReLU
     ↓
MaxPooling
2 × 2
     ↓
Conv2D
64 filters, 3 × 3
ReLU
     ↓
MaxPooling
2 × 2
     ↓
Flatten
1600 features
     ↓
Dense
128 neurons
ReLU
     ↓
Dropout
50%
     ↓
Dense
10 neurons
Softmax
     ↓
Output
10 Classes
```

---

13. Layer-by-Layer Summary

Layer	Configuration	Main Purpose
Input	`28 × 28 × 1`	Receives grayscale image
Conv2D	`32 filters, 3 × 3`	Extracts low-level features
MaxPooling2D	`2 × 2`	Reduces spatial dimensions
Conv2D	`64 filters, 3 × 3`	Extracts higher-level features
MaxPooling2D	`2 × 2`	Further reduces feature maps
Flatten	—	Converts features to 1D
Dense	`128 neurons`	Learns feature combinations
Dropout	`0.5`	Reduces overfitting
Dense	`10 neurons`	Produces class probabilities

---

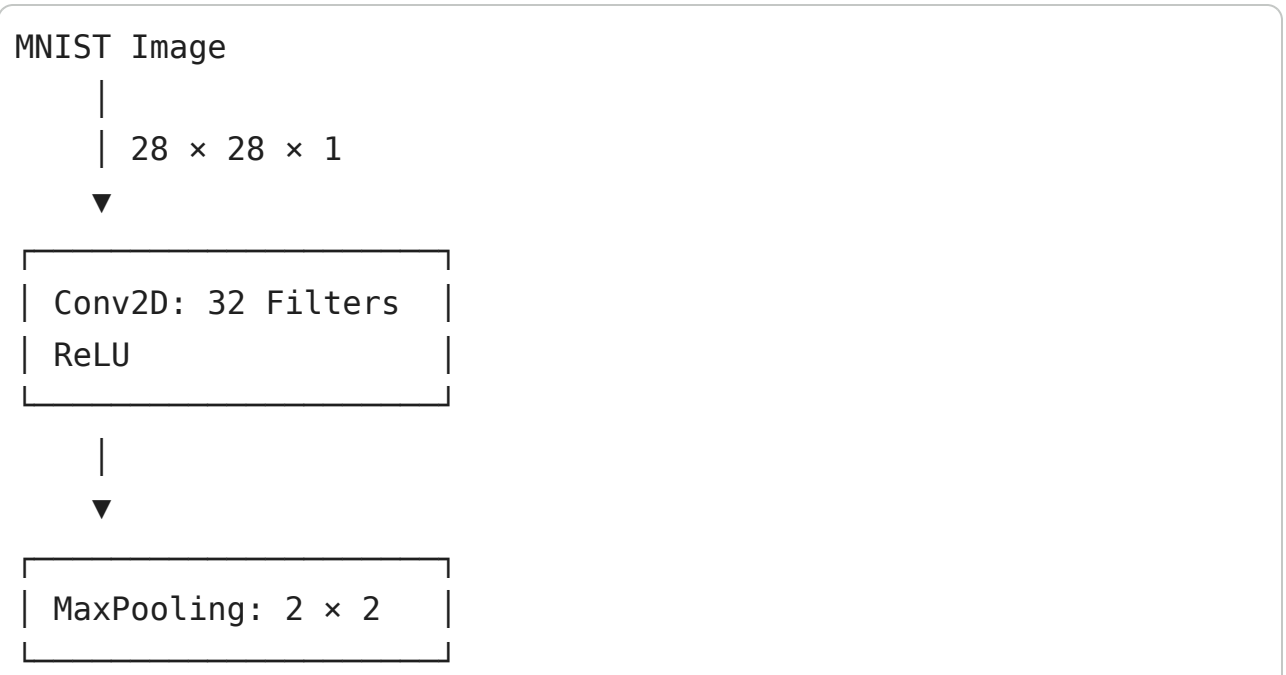
14. Why This Architecture Is Suitable for MNIST

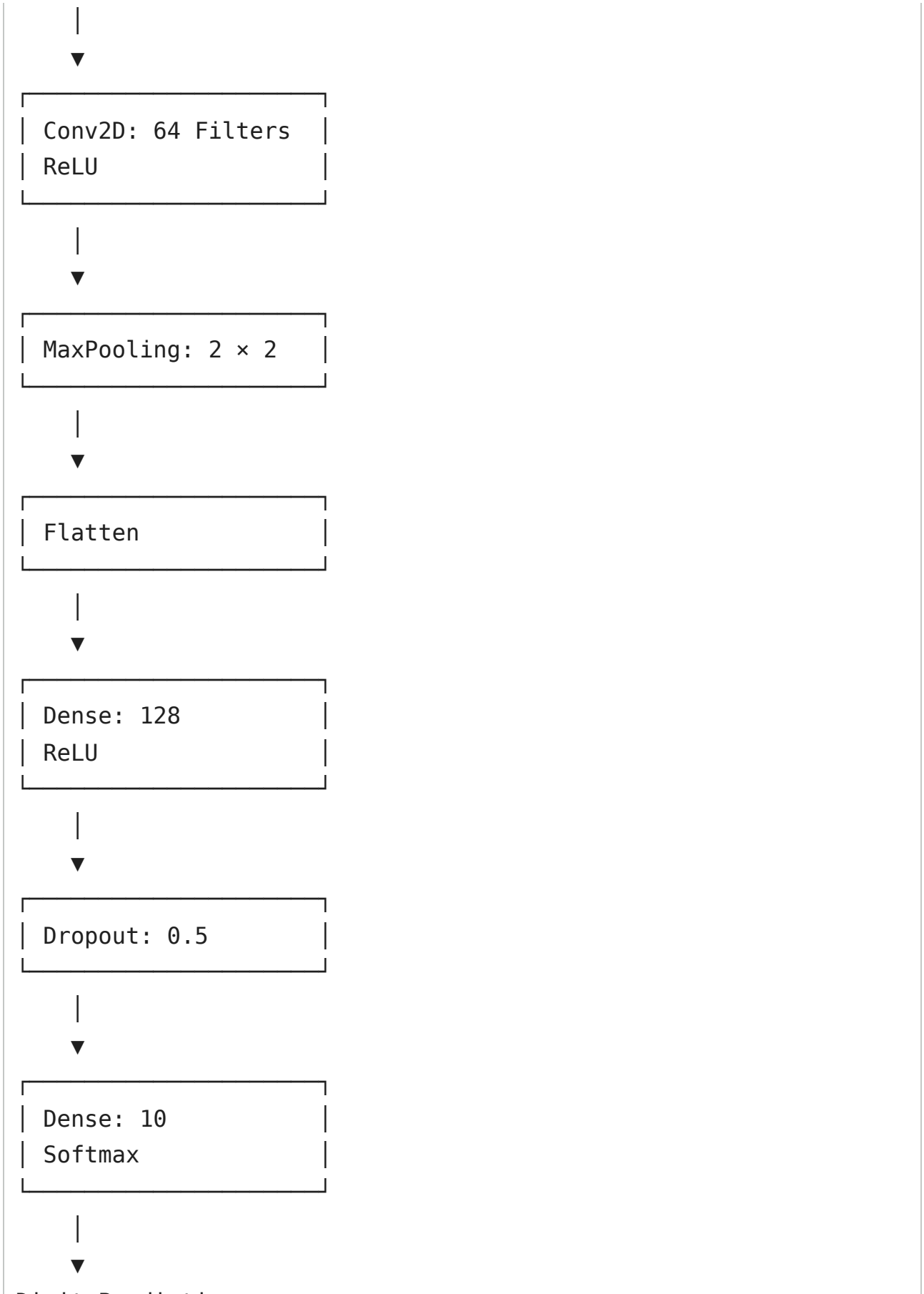
This CNN architecture is suitable for MNIST because:

1. **Convolutional layers** automatically learn important image features.
2. **Max pooling** reduces the spatial dimensions and computational cost.
3. **Flattening** converts extracted image features into a format suitable for dense layers.
4. **Dense layers** perform the final classification.
5. **Dropout** helps reduce overfitting.
6. **Softmax** produces probabilities for the ten digit classes.

---

15. Overall Data Flow




Conv2D: 64 Filters
ReLU
MaxPooling: 2 × 2
Flatten
Dense: 128
ReLU
Dropout: 0.5
Dense: 10
Softmax
Digit Prediction
0 – 9

16. Conclusion

The CNN model uses a combination of **convolutional layers, pooling layers, a flattening layer, fully connected layers, dropout, and a softmax output layer**.

The convolutional layers are responsible for extracting visual features from handwritten digits, while the dense layers use those features to determine which of the **10 digit classes** the input image belongs to.

This architecture provides a simple yet effective deep learning model for **MNIST handwritten digit classification**.

```
# Build CNN Model
model = Sequential([

    Conv2D(32, 3, activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, 3, activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dropout(0.5),

    Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/ba
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Displaying the CNN Model Summary

After building the CNN architecture, the `model.summary()` function can be used to display a detailed summary of the model.

Code

```
model.summary()
```

---

1. Purpose of `model.summary()`

The `model.summary()` function displays important information about the CNN architecture, including:

- The name of each layer.
- The output shape of each layer.
- The number of trainable parameters.
- The total number of parameters.
- The number of non-trainable parameters.

It is useful for verifying whether the model architecture has been constructed correctly.

---

2. Expected Model Summary

For the CNN architecture defined above, the output will be approximately:

```
Model: "sequential"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━
┃ Layer (type)                    ┃ Output Shape           ┃ Param #
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━
│ conv2d (Conv2D)                 │ (None, 26, 26, 32)     │ 320
├─────────────────────────────────┼────────────────────────┼──────────
│ max_pooling2d (MaxPooling2D)    │ (None, 13, 13, 32)     │ 0
├─────────────────────────────────┼────────────────────────┼──────────
│ conv2d_1 (Conv2D)               │ (None, 11, 11, 64)     │ 18,496
├─────────────────────────────────┼────────────────────────┼──────────
│ max_pooling2d_1 (MaxPooling2D)  │ (None, 5, 5, 64)       │ 0
├─────────────────────────────────┼────────────────────────┼──────────
│ flatten (Flatten)               │ (None, 1600)           │ 0
├─────────────────────────────────┼────────────────────────┼──────────
│ dense (Dense)                   │ (None, 128)            │ 204,928
├─────────────────────────────────┼────────────────────────┼──────────
│ dropout (Dropout)               │ (None, 128)            │ 0
├─────────────────────────────────┼────────────────────────┼──────────
│ dense_1 (Dense)                 │ (None, 10)             │ 1,290
└─────────────────────────────────┴────────────────────────┴──────────

Total params: 225,034
```

```
Trainable params: 225,034
Non-trainable params: 0
```

> **Note:** The exact layer names can vary depending on the TensorFlow/Keras version and whether other models have already been created. The parameter counts and output dimensions for this architecture remain the same.

---

3. Understanding the Output Shape

The **Output Shape** column shows the shape of the data produced by each layer.

First Convolutional Layer

```
(None, 26, 26, 32)
```

Here:

- `None` → Batch size, determined automatically.
- `26 × 26` → Spatial dimensions after the 3 × 3 convolution.
- `32` → Number of filters.

The spatial dimension becomes 26 because the default padding is `valid`:

```
28 - 3 + 1 = 26
```

First Max Pooling Layer

```
(None, 13, 13, 32)
```

The `2 × 2` max-pooling operation reduces:

```
26 × 26 → 13 × 13
```

The number of feature maps remains `32`.

Second Convolutional Layer

```
(None, 11, 11, 64)
```

The second 3 × 3 convolution changes:

```
13 × 13 → 11 × 11
```

The number of feature maps increases from:

```
32 → 64
```

Second Max Pooling Layer

```
(None, 5, 5, 64)
```

The second pooling operation reduces:

```
11 × 11 → 5 × 5
```

The number of channels remains `64`.

Flatten Layer

```
(None, 1600)
```

The feature maps:

```
5 × 5 × 64
```

are converted into a one-dimensional vector:

```
5 × 5 × 64 = 1600
```

Therefore, each image produces **1,600 features**.

Dense Layer

```
(None, 128)
```

The 1,600 extracted features are connected to **128 neurons**.

This layer learns combinations of the features that are useful for recognizing handwritten digits.

Dropout Layer

```
(None, 128)
```

The dropout layer does not change the shape of the data.

It only randomly deactivates a portion of neurons during training to help reduce overfitting.

Output Layer

```
(None, 10)
```

The final layer contains **10 neurons**, representing the ten MNIST classes:

```
0  1  2  3  4  5  6  7  8  9
```

The `softmax` activation produces a probability for each class.

---

4. Understanding the Parameter Count

The `Param #` column shows how many parameters each layer needs to learn.

First Conv2D Layer

```
320 parameters
```

The calculation is:

```
(3 × 3 × 1 + 1) × 32
= 320
```

The `1` represents the bias term.

Second Conv2D Layer

```
18,496 parameters
```

Calculation:

```
(3 × 3 × 32 + 1) × 64
= 18,496
```

The second convolution receives 32 input channels and produces 64 output channels.

First Dense Layer

```
204,928 parameters
```

Calculation:

```
(1600 × 128) + 128
= 204,928
```

The additional `128` represents the bias values.

Output Dense Layer

```
1,290 parameters
```

Calculation:

```
(128 × 10) + 10
= 1,290
```

---

5. Total Parameters

The total number of parameters is:

```
320 + 18,496 + 204,928 + 1,290
= 225,034
```

Therefore:

```
Total parameters: 225,034
```

All of these parameters are trainable in this model.

```
Trainable parameters: 225,034
Non-trainable parameters: 0
```

---

6. Meaning of `None`

You may notice that every output shape begins with:

```
None
```

For example:

```
(None, 26, 26, 32)
```

`None` represents the **batch size**.

The model does not require a fixed number of images at once.

For example, it can process:

```
32 images
64 images
128 images
```

depending on the batch size used during training.

Therefore, `None` means that the batch size is determined dynamically.

---

7. Why `model.summary()` Is Important

Using `model.summary()` helps verify:

1. Whether all required layers are present.
2. Whether the output dimensions are correct.
3. Whether the CNN architecture is compatible with the input data.
4. How many parameters the model contains.
5. Which layers contribute most of the parameters.
6. Whether parameters are trainable or non-trainable.

---

8. Complete Code

```
# Display CNN model architecture
model.summary()
```

This command does not train the model. It only **displays information about the architecture**.

---

9. Summary

Layer	Output Shape	Parameters
Conv2D	`(None, 26, 26, 32)`	320
MaxPooling2D	`(None, 13, 13, 32)`	0
Conv2D	`(None, 11, 11, 64)`	18,496
MaxPooling2D	`(None, 5, 5, 64)`	0
Flatten	`(None, 1600)`	0
Dense	`(None, 128)`	204,928
Dropout	`(None, 128)`	0
Dense	`(None, 10)`	1,290
Total	—	**225,034**

Conclusion

The `model.summary()` command provides a compact overview of the CNN architecture. For this MNIST classifier, the model contains **225,034 trainable parameters** and progressively transforms the original `28 × 28 × 1` image into a final prediction across **10 digit classes**.

```
1 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param
conv2d (Conv2D)	(None, 26, 26, 32)	32
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,49
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
flatten (Flatten)	(None, 1600)	
dense (Dense)	(None, 128)	204,92
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 10)	1,29

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

Compiling the CNN Model

After defining the CNN architecture, the model must be **compiled** before it can be trained.

Compilation specifies:

- The optimization algorithm.
- The loss function.
- The evaluation metrics.

The following code compiles the CNN model:

```
# Compile Model

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

---

1. Purpose of Model Compilation

The `model.compile()` function prepares the neural network for training.

During compilation, we tell the model:

1. **How to update its weights** → `optimizer`
2. **How to measure prediction error** → `loss`
3. **How to evaluate performance** → `metrics`

The model cannot normally be trained with `model.fit()` until it has been compiled.

---

2. Optimizer — Adam

```
optimizer='adam'
```

The **Adam (Adaptive Moment Estimation)** optimizer is used to update the model's trainable parameters during training.

The optimizer determines how the model changes its weights based on the calculated error.

Why Adam?

Adam is widely used in deep learning because it:

- Adapts the learning rate during training.
- Generally converges efficiently.
- Works well for many neural network architectures.
- Requires relatively little manual configuration.

During training, the process is approximately:

```
Input Image
     ↓
CNN Prediction
     ↓
Calculate Error
     ↓
Backpropagation
     ↓
Adam Updates Weights
     ↓
Improved Prediction
```

This process is repeated for many training examples and epochs.

---

3. Loss Function

```
loss='sparse_categorical_crossentropy'
```

The **sparse categorical cross-entropy** loss function is used to measure how different the model's predicted probabilities are from the correct class labels.

This loss function is appropriate when:

- There are multiple classes.
- Each sample belongs to exactly one class.
- The labels are stored as integer class IDs.

For MNIST, the classes are:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

For example:

```
y_train[0] = 5
```

The label is represented by the integer `5`, rather than a one-hot vector such as:

```
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
```

Therefore, `sparse_categorical_crossentropy` is a suitable choice.

---

4. Why Sparse Categorical Cross-Entropy?

There are two commonly used categorical cross-entropy approaches:

Sparse Categorical Cross-Entropy

Used when labels are integers:

```
0, 1, 2, ..., 9
```

Example:

```
y_train = [5, 0, 4, 1, 9, ...]
```

Categorical Cross-Entropy

Used when labels are one-hot encoded:

```
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
```

Since MNIST labels are normally integer values, the following is appropriate:

```
loss='sparse_categorical_crossentropy'
```

---

5. Accuracy Metric

```
metrics=['accuracy']
```

The `accuracy` metric is used to measure how many predictions are correct.

For example, suppose the model makes 100 predictions and correctly classifies 95 of them.

Then:

```
Accuracy = 95 / 100
         = 0.95
         = 95%
```

Accuracy provides an easy-to-understand measure of the model's classification performance.

---

6. Understanding the Three Components

The compilation configuration can be summarized as:

Component	Selected Value	Purpose
Optimizer	`adam`	Updates model weights
Loss	`sparse_categorical_crossentropy`	Measures prediction error
Metric	`accuracy`	Measures classification performance

---

7. Complete Compilation Code

```
# Compile Model

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

After this step, the CNN model is **compiled and ready for training**.

The next step is typically to use:

```
model.fit()
```

to train the model using the training dataset.

---

8. Overall Training Concept

The compilation settings define how the learning process will work:

```
            Training Image
                  ↓
             CNN Model
                  ↓
             Prediction
                  ↓
         Loss Calculation
                  ↓
           Backpropagation
                  ↓
            Adam Optimizer
                  ↓
            Update Weights
                  ↓
         Improved CNN Model
```

The `accuracy` metric is used to monitor how well the model is performing while this process takes place.

---

9. Conclusion

The `model.compile()` function configures the CNN before training.

In this project:

- **Adam** is used as the optimizer.
- **Sparse categorical cross-entropy** is used as the loss function because MNIST labels are integer class IDs.
- **Accuracy** is used to measure classification performance.

Therefore, the compiled model is now ready to learn how to classify handwritten digits from **0 to 9**.

```
# Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Early Stopping Callback

During neural network training, the model may continue learning even after its performance on unseen validation data stops improving. This can lead to **overfitting**.

To prevent unnecessary training and reduce overfitting, an **Early Stopping callback** is used.

The following code defines the Early Stopping callback:

```
# Early Stopping Callback

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
```

```
    verbose=1
)
```

---

1. Purpose of Early Stopping

Early stopping automatically stops model training when the monitored validation metric stops improving.

In this project, the callback monitors:

```
monitor='val_loss'
```

Therefore, the model watches the **validation loss** after every epoch.

The basic idea is:

```
Training
   ↓
Epoch 1 → Validation Loss
   ↓
Epoch 2 → Validation Loss improves
   ↓
Epoch 3 → Validation Loss improves
   ↓
Epoch 4 → No improvement
   ↓
...
   ↓
3 consecutive epochs without improvement
   ↓
Training stops
```

This prevents the model from unnecessarily continuing to train.

---

2. `EarlyStopping()`

```
early_stop = EarlyStopping(...)
```

`EarlyStopping` is a callback provided by Keras.

A **callback** is a mechanism that allows Keras to perform a specific action automatically during model training.

In this case, the callback monitors validation performance and can stop training when further training is no longer beneficial.

---

3. Monitoring `val_loss`

```
monitor='val_loss'
```

The `monitor` parameter specifies which value should be monitored.

Here:

```
val_loss
```

means **validation loss**.

During training, the model calculates loss on the validation dataset after each epoch.

For example:

```
Epoch 1 → val_loss = 0.120
Epoch 2 → val_loss = 0.095
Epoch 3 → val_loss = 0.080
Epoch 4 → val_loss = 0.078
Epoch 5 → val_loss = 0.081
```

The goal is generally to minimize validation loss.

Therefore, lower `val_loss` values indicate better validation performance.

---

4. Patience

```
patience=3
```

The `patience` parameter specifies how many consecutive epochs the model is allowed to continue without improvement before training is stopped.

Here:

```
patience = 3
```

means the model can tolerate **3 consecutive epochs without improvement**.

For example:

```
Epoch 1 → val_loss = 0.100  ✓ Improved
Epoch 2 → val_loss = 0.085  ✓ Improved
Epoch 3 → val_loss = 0.083  ✓ Improved
Epoch 4 → val_loss = 0.084  ✗ No improvement
Epoch 5 → val_loss = 0.086  ✗ No improvement
Epoch 6 → val_loss = 0.087  ✗ No improvement
Epoch 7 → Training stops
```

The exact stopping epoch can depend on when the callback determines that the patience condition has been reached.

---

5. Restoring the Best Weights

```
restore_best_weights=True
```

This is an important option.

During training, the model's weights change after every epoch. The model may achieve its best validation performance at an earlier epoch and then start getting worse.

When:

```
restore_best_weights=True
```

is used, Keras restores the model weights from the epoch that produced the **best monitored value**.

For example:

```
Epoch 1 → val_loss = 0.120
Epoch 2 → val_loss = 0.090  ← Best
Epoch 3 → val_loss = 0.095
Epoch 4 → val_loss = 0.110
Epoch 5 → val_loss = 0.115
```

When training stops, the model restores the weights from:

```
Epoch 2
```

because it had the lowest validation loss.

---

6. Verbose Mode

```
verbose=1
```

The `verbose` parameter controls whether Keras displays information about the callback's actions.

With:

```
verbose=1
```

Keras displays a message when training is stopped.

For example, you may see a message similar to:

```
Epoch 7: early stopping
```

The exact output can vary depending on the Keras version.

---

7. Complete Callback Configuration

The callback can be summarized as:

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)
```

Parameter	Value	Purpose
`monitor`	`'val_loss'`	Monitors validation loss
`patience`	`3`	Allows 3 epochs without improvement

Parameter	Value	Purpose
`restore_best_weights`	`True`	Restores the best model weights
`verbose`	`1`	Displays early stopping information

---

8. Why Early Stopping Is Useful

Early stopping provides several benefits.

Reduces Overfitting

If validation performance stops improving while training continues, the model may begin to overfit the training data.

Early stopping helps prevent excessive training.

Saves Training Time

There is no need to complete all predefined epochs if the model has already stopped improving.

Preserves the Best Model

With:

```
restore_best_weights=True
```

the model keeps the weights associated with the best validation loss.

Improves Training Efficiency

The model automatically determines when further training is unlikely to provide additional validation improvement.

---

9. Early Stopping with Model Training

The callback is not executed simply by defining it.

It must be passed to the `model.fit()` function.

For example:

```
history = model.fit(
    X_train,
    y_train,
```

```
    epochs=20,
    batch_size=64,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
```

Here, the callback:

```
callbacks=[early_stop]
```

is passed to the training process.

The training process can then be summarized as:

```
Training Data
     ↓
CNN Model
     ↓
Epoch Training
     ↓
Validation Loss
     ↓
Has Validation Loss Improved?
     |
  ┌──┴──────┐
Yes       No
 |         |
Continue  Check Patience
           |
       3 epochs without
          improvement
           |
           ↓
      Stop Training
           |
           ↓
Restore Best Weights
```

10. Example Training Scenario

Suppose the model is configured for:

```
epochs=20
```

This means the maximum number of training epochs is 20.

However, early stopping may stop training earlier.

For example:

```
Epoch 1  → val_loss improves
Epoch 2  → val_loss improves
Epoch 3  → val_loss improves
Epoch 4  → no improvement
Epoch 5  → no improvement
Epoch 6  → no improvement
Epoch 7  → Early stopping
```

Although the maximum was 20 epochs, the model may stop before reaching that number.

The best weights are then restored.

---

11. Important Distinction

Early stopping does **not** change the CNN architecture.

It only controls the **training process**.

The CNN architecture remains:

```
Input
  ↓
Conv2D
  ↓
MaxPooling
  ↓
Conv2D
  ↓
```

```
MaxPooling
  ↓
Flatten
  ↓
Dense
  ↓
Dropout
  ↓
Dense
  ↓
Output
```

Early stopping determines **when to stop training this architecture**.

---

12. Conclusion

The Early Stopping callback is an important training technique used to prevent unnecessary training and reduce the risk of overfitting.

In this project:

- `val_loss` is monitored.
- Training can tolerate 3 epochs without improvement.
- The best model weights are restored.
- A message is displayed when early stopping occurs.

The callback therefore helps ensure that the final CNN model corresponds to the point where it achieved its **best validation performance**, rather than simply the last epoch completed.

```
# Early Stopping Callback
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)
```

Training the CNN Model

After building and compiling the CNN model and defining the Early Stopping callback, the next step is to **train the model** using the MNIST training dataset.

The `model.fit()` function is used to train the CNN.

Code

```
# Train Model

history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=64,

    # Automatically creates validation set
    validation_split=0.2,

    callbacks=[early_stop],

    verbose=1
)
```

---

1. Purpose of `model.fit()`

The `model.fit()` function trains the neural network using the provided training images and their corresponding labels.

During training, the model:

1. Receives input images.
2. Makes predictions.
3. Calculates the prediction error.
4. Performs backpropagation.
5. Updates its weights using the Adam optimizer.
6. Repeats this process for multiple epochs.

The overall process is:

```
Training Images + Labels
          ↓
       CNN Model
          ↓
      Predictions
          ↓
     Calculate Loss
          ↓
    Backpropagation
          ↓
     Update Weights
          ↓
   Improved Model
```

---

2. Training Images — `X_train`

```
X_train
```

`X_train` contains the input images used for training the CNN.

After preprocessing, the training images have the shape:

```
(60000, 28, 28, 1)
```

This represents:

- `60000` → Number of training images
- `28` → Image height
- `28` → Image width
- `1` → Grayscale channel

The model uses these images to learn visual patterns associated with each handwritten digit.

---

3. Training Labels — `y_train`

```
y_train
```

`y_train` contains the correct class labels corresponding to the training images.

The labels range from:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

For example:

```
X_train[0] → Image of a handwritten digit
y_train[0] → Correct digit label
```

The model compares its prediction with `y_train` to calculate the loss.

---

4. Number of Epochs

```
epochs=20
```

An **epoch** represents one complete pass through the training data.

Here:

```
epochs = 20
```

means the model is allowed to train for a maximum of **20 epochs**.

For example:

```
Epoch 1 → Complete training dataset
Epoch 2 → Complete training dataset
Epoch 3 → Complete training dataset
...
Epoch 20 → Complete training dataset
```

However, because Early Stopping is enabled, the model may stop before reaching 20 epochs if the validation loss stops improving.

---

5. Batch Size

```
batch_size=64
```

The `batch_size` specifies how many training samples are processed before the model updates its weights.

Here:

```
batch size = 64
```

means the model processes **64 images at a time**.

For example:

```
Training Dataset
     ↓
64 images
     ↓
Prediction + Loss
     ↓
Weight Update
     ↓
Next 64 images
     ↓
...
```

Using batches makes training more computationally efficient than processing the entire dataset at once.

---

6. Validation Split

```
validation_split=0.2
```

The `validation_split` parameter automatically reserves a portion of the training data for validation.

Here:

```
validation_split = 0.2
```

means **20% of the provided training data** is used for validation.

With 60,000 training images:

```
Training data = 60,000 images
Validation = 20% of 60,000
```

Therefore:

```
Validation images = 12,000
Training images   = 48,000
```

Conceptually:

```
X_train
60,000 images
     |
     ├──────────────┐
     |              |
     ▼              ▼
80% Training       20% Validation
48,000 images      12,000 images
```

The validation data is used to monitor how well the model performs on data that was not directly used to update its weights.

---

7. Why Validation Data Is Important

Validation data helps determine whether the model is generalizing well.

For example:

```
Training Accuracy ↑
Validation Accuracy ↑
```

generally indicates that the model is learning useful patterns.

However, if:

```
Training Accuracy ↑↑
Validation Accuracy ↓
```

the model may be **overfitting**.

The validation loss is also monitored by the Early Stopping callback:

```
monitor='val_loss'
```

Therefore, the validation set plays an important role in determining when training should stop.

---

8. Early Stopping Callback

```
callbacks=[early_stop]
```

This passes the previously defined Early Stopping callback to the training process.

The callback monitors:

```
val_loss
```

and stops training when the validation loss stops improving for the specified patience period.

The callback was defined as:

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)
```

Therefore, even though:

```
epochs=20
```

is specified, the model may train for fewer than 20 epochs.

---

9. Verbose Mode

```
verbose=1
```

The `verbose` parameter controls the amount of information displayed during training.

With:

```
verbose=1
```

Keras displays a progress bar and training information for each epoch.

Typical output may include:

```
Epoch 1/20
750/750 ━━━━━━━━━━━━━━━━━━━━
loss: 0.1500
accuracy: 0.9540
val_loss: 0.0600
val_accuracy: 0.9810
```

The exact output depends on the training process and Keras version.

---

10. The `history` Variable

```
history = model.fit(...)
```

The result of `model.fit()` is stored in the variable:

```
history
```

This object contains the training history.

It stores values such as:

- Training loss
- Training accuracy
- Validation loss
- Validation accuracy

These values can later be used to create graphs showing model performance over time.

For example:

```
history.history.keys()
```

may contain:

```
loss
accuracy
val_loss
val_accuracy
```

---

11. Training and Validation Metrics

During training, four important metrics are typically available.

Training Loss

```
loss
```

Measures the error of the model on the training data.

Training Accuracy

```
accuracy
```

Measures the percentage of correctly classified training images.

Validation Loss

```
val_loss
```

Measures the error on the validation dataset.

Validation Accuracy

```
val_accuracy
```

Measures the classification accuracy on the validation dataset.

These values can be used to analyze whether the model is learning effectively.

---

12. Example Training Process

A simplified example of training output might look like:

```
Epoch 1/20
loss: 0.1800
accuracy: 0.9450
val_loss: 0.0700
val_accuracy: 0.9780

Epoch 2/20
loss: 0.0650
accuracy: 0.9810
val_loss: 0.0500
val_accuracy: 0.9850

Epoch 3/20
loss: 0.0450
accuracy: 0.9860
val_loss: 0.0480
val_accuracy: 0.9860

...
```

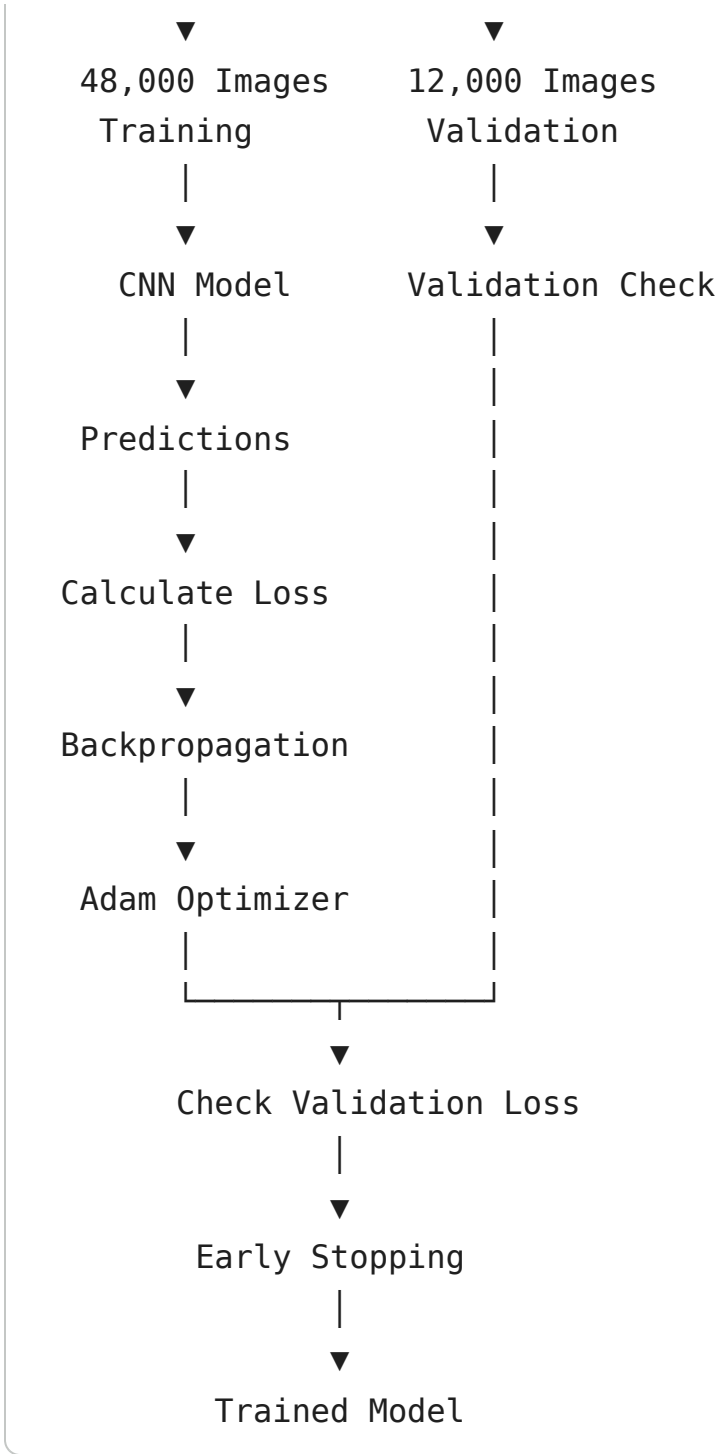
If validation loss stops improving for the specified patience period, Early Stopping terminates the training process.

---

13. Complete Training Pipeline

The training process can be summarized as:

```
MNIST Training Dataset
        │
        ▼
    60,000 Images
        │
        ▼
   validation_split=0.2
        │
        ├──────────────────┐
```



---

14. Important Parameters Summary

Parameter	Value	Purpose
`X_train`	Training images	Input data
`y_train`	Training labels	Target classes
`epochs`	`20`	Maximum number of training cycles
`batch_size`	`64`	Images processed per weight update
`validation_split`	`0.2`	Uses 20% of training data for validation

Parameter	Value	Purpose
`callbacks`	`[early_stop]`	Enables Early Stopping
`verbose`	`1`	Displays training progress

---

15. Why `model.fit()` Is Important

The CNN architecture only defines the structure of the neural network. It does not automatically learn anything.

The `model.fit()` function is responsible for actually **training the network**.

Before training:

```
CNN
↓
Random/Initial Weights
```

After training:

```
CNN
↓
Learned Weights
↓
Can Recognize Handwritten Digits
```

Therefore, `model.fit()` is the main step where the CNN learns from the MNIST training data.

---

16. Conclusion

The `model.fit()` function trains the CNN using the MNIST dataset.

In this project:

- The model is trained for a maximum of **20 epochs**.
- **64 images** are processed per batch.
- **20% of the training data** is automatically used for validation.
- **Early Stopping** monitors validation loss and can stop training early.
- Training history is stored in the `history` variable.
- Training and validation accuracy/loss can later be visualized to evaluate the learning process.

After this step, the CNN model is trained and ready to be evaluated on the separate **MNIST test dataset**.

```
# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=64,

    # Automatically creates validation set
    validation_split=0.2,

    callbacks=[early_stop],

    verbose=1
)
```

```
Epoch 1/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 45s 57ms/step - accuracy: 0.9164 - loss: 0.2
Epoch 2/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 40s 53ms/step - accuracy: 0.9725 - loss: 0.0
Epoch 3/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 55ms/step - accuracy: 0.9791 - loss: 0.0
Epoch 4/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.9835 - loss: 0.0
Epoch 5/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 81s 55ms/step - accuracy: 0.9850 - loss: 0.0
Epoch 6/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 82s 55ms/step - accuracy: 0.9881 - loss: 0.0
Epoch 7/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 55ms/step - accuracy: 0.9887 - loss: 0.0
Epoch 8/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.9906 - loss: 0.0
Epoch 9/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.9912 - loss: 0.0
Epoch 10/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 55ms/step - accuracy: 0.9914 - loss: 0.0
Epoch 11/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.9926 - loss: 0.0
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 8.
```

Evaluating the CNN Model

After training the CNN model, the next step is to evaluate its performance on the **test dataset**.

The test dataset contains images that were not used during model training. Therefore, it provides an independent measurement of how well the trained model can classify unseen handwritten digits.

The following code evaluates the trained model:

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("\nTest Accuracy:", test_accuracy)
print("Test Loss:", test_loss)
```

---

1. Purpose of Model Evaluation

The purpose of `model.evaluate()` is to measure the performance of the trained CNN on the test dataset.

The model calculates two important values:

- **Test Loss**
- **Test Accuracy**

These values help determine how well the model generalizes to new, unseen images.

---

2. Using `model.evaluate()`

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

The `evaluate()` function takes two main inputs:

```
X_test
y_test
```

where:

- `X_test` contains the test images.
- `y_test` contains the correct labels for those images.

The model compares its predictions against the actual labels and calculates the configured loss and accuracy.

---

3. Test Dataset

`X_test`

```
X_test
```

contains the MNIST test images.

After preprocessing and reshaping, its shape is:

```
(10000, 28, 28, 1)
```

This represents:

- `10000` → Number of test images
- `28` → Image height
- `28` → Image width
- `1` → Grayscale channel

`y_test`

```
y_test
```

contains the correct labels for the test images.

The labels range from:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

---

4. Test Loss

The first returned value is stored in:

```
test_loss
```

Test loss represents the error between the model's predictions and the actual test labels.

A lower loss generally indicates that the model's predictions are closer to the correct answers.

For example:

```
Test Loss: 0.045
```

would indicate a relatively small prediction error.

The exact value depends on the trained model.

---

5. Test Accuracy

The second returned value is stored in:

```
test_accuracy
```

Test accuracy represents the proportion of test images that the model classified correctly.

For example, if the model correctly classifies 9,800 out of 10,000 test images:

```
Accuracy = 9800 / 10000
         = 0.98
         = 98%
```

Therefore, a test accuracy of:

```
0.98
```

corresponds to:

```
98%
```

---

6. Printing Test Accuracy

```
print("\nTest Accuracy:", test_accuracy)
```

This prints the accuracy achieved by the CNN on the test dataset.

The `\n` creates a new line before the output, making the result easier to read.

Example output:

```
Test Accuracy: 0.985
```

This means the model correctly classified approximately **98.5% of the test images**.

---

7. Printing Test Loss

```
print("Test Loss:", test_loss)
```

This displays the loss calculated on the test dataset.

Example:

```
Test Loss: 0.045
```

A lower test loss generally indicates better predictive performance.

---

8. Example Output

The actual values depend on the training process, but the output may look similar to:

```
313/313 ━━━━━━━━━━━━━━━━━━━━
Test Accuracy: 0.9854
Test Loss: 0.0472
```

The first line shows the evaluation progress.

The next two lines show:

```
Test Accuracy → 0.9854
Test Loss     → 0.0472
```

The accuracy can also be expressed as:

```
0.9854 × 100 = 98.54%
```

So the model correctly classifies approximately **98.54%** of the test images.

---

9. Training Accuracy vs Test Accuracy

It is important to distinguish between training and test performance.

Training Accuracy

Training accuracy measures performance on data used to train the model.

Validation Accuracy

Validation accuracy measures performance on the validation portion of the training data.

Test Accuracy

Test accuracy measures performance on the separate test dataset.

The overall process is:

```
Training Data
     ↓
Model Training
     ↓
Validation Data
     ↓
Model Selection / Early Stopping
     ↓
Final Trained Model
     ↓
Test Data
     ↓
Final Performance Evaluation
```

Test accuracy is especially important because the test images were not used to update the model's weights.

---

10. Why Test Evaluation Is Important

Evaluating the model on the test dataset helps determine whether the CNN can **generalize** beyond the images it saw during training.

A model that performs well on training data but poorly on test data may be suffering from **overfitting**.

For example:

```
Training Accuracy → 99.9%
Test Accuracy     → 92%
```

This large difference could indicate that the model has learned the training data too specifically.

On the other hand:

```
Training Accuracy → 98.5%
Test Accuracy     → 98.2%
```

suggests that the model is generalizing well.

---

11. Complete Evaluation Code

```
# Evaluate Model

test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("\nTest Accuracy:", test_accuracy)
print("Test Loss:", test_loss)
```

This code should be executed **after the model has been trained**.

---

12. Overall CNN Workflow

The complete workflow developed so far can be summarized as:

```
Load MNIST Dataset
        ↓
Visualize Images
        ↓
Normalize Pixel Values
        ↓
Reshape Images for CNN
        ↓
Build CNN Model
        ↓
```

```
Display Model Summary
        ↓
Compile Model
        ↓
Configure Early Stopping
        ↓
Train Model
        ↓
Evaluate on Test Dataset
        ↓
Test Accuracy & Test Loss
```

---

13. Summary

Code / Variable	Purpose
`model.evaluate()`	Evaluates the trained model
`X_test`	Contains unseen test images
`y_test`	Contains correct test labels
`test_loss`	Measures prediction error
`test_accuracy`	Measures correct classification rate
`print()`	Displays evaluation results

Conclusion

The `model.evaluate()` function provides the final performance measurement of the trained CNN on the MNIST test dataset.

The **test accuracy** indicates how successfully the model recognizes unseen handwritten digits, while the **test loss** measures the model's prediction error.

Therefore, this evaluation step is essential for determining the **generalization performance and effectiveness** of the trained CNN model.

```
1 test_loss, test_accuracy = model.evaluate(X_test, y_test)
2
3 print("\nTest Accuracy:", test_accuracy)
4 print("Test Loss:", test_loss)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9911 - loss: 0.02
```

```
    Test Accuracy: 0.991100013256073
    Test Loss: 0.02756749838590622
```

Plotting Training and Validation Accuracy

After training the CNN model, the training history stored in the `history` object can be used to visualize how the model's accuracy changed over the training epochs.

The following code creates a graph comparing **Training Accuracy** and **Validation Accuracy**.

Code

```
# Plot Accuracy

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```

---

1. Purpose of the Accuracy Plot

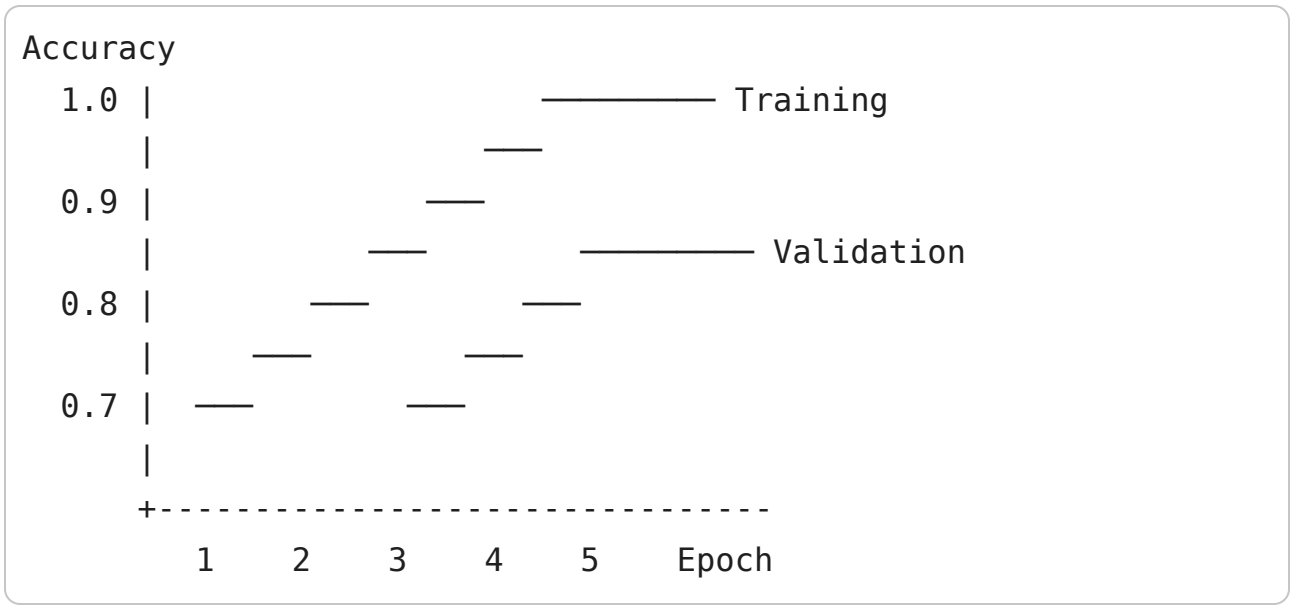
The accuracy plot helps us understand how well the CNN learns during training.

It compares two values:

- **Training Accuracy**
- **Validation Accuracy**

The graph shows how these values change across different epochs.

A typical graph may look conceptually like:



The actual curves depend on the training results.

---

2. Creating the Figure

```
plt.figure(figsize=(8, 5))
```

This creates a new figure for the accuracy graph.

The parameter:

```
figsize=(8, 5)
```

specifies the figure size:

- Width = 8 inches
- Height = 5 inches

A larger figure makes the graph easier to read.

---

3. Plotting Training Accuracy

```
plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)
```

The training accuracy values are stored inside:

```
history.history['accuracy']
```

These values represent the model's accuracy on the training data after each epoch.

For example:

```
Epoch 1 → 0.94
Epoch 2 → 0.96
Epoch 3 → 0.97
Epoch 4 → 0.98
```

The `label` parameter gives this line the name:

```
Training Accuracy
```

This name will appear in the graph legend.

---

4. Plotting Validation Accuracy

```
plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)
```

The validation accuracy values are stored in:

```
history.history['val_accuracy']
```

These values represent the model's performance on the validation dataset after each epoch.

Because the training code used:

```
validation_split=0.2
```

20% of the training data was used for validation.

The validation accuracy helps us understand how well the model generalizes to data that was not used directly to update the model's weights.

---

5. Adding a Graph Title

```
plt.title("Training vs Validation Accuracy")
```

This gives the graph a descriptive title.

The title clearly indicates that the graph compares:

```
Training Accuracy
        vs
Validation Accuracy
```

---

6. X-Axis Label

```
plt.xlabel("Epoch")
```

The x-axis represents the **epoch number**.

An epoch is one complete pass through the training dataset.

For example:

```
1 → First epoch
2 → Second epoch
3 → Third epoch
...
20 → Maximum possible epoch
```

Because Early Stopping is enabled, the graph may contain fewer than 20 points if training stops early.

---

7. Y-Axis Label

```
plt.ylabel("Accuracy")
```

The y-axis represents the accuracy achieved by the model.

Accuracy is generally represented as a value between:

```
0 and 1
```

For example:

```
0.90 = 90%
0.95 = 95%
0.98 = 98%
```

---

8. Adding the Legend

```
plt.legend()
```

The legend identifies the two curves in the graph.

It allows us to distinguish between:

```
Training Accuracy
Validation Accuracy
```

Without a legend, it would be difficult to determine which curve represents which dataset.

---

9. Adding Grid Lines

```
plt.grid(True)
```

This adds grid lines to the graph.

Grid lines make it easier to compare accuracy values and identify changes across epochs.

10. Displaying the Graph

```
plt.show()
```

The `plt.show()` function displays the completed graph.

Without this command, the figure may not be displayed in some Python environments.

---

11. Understanding the Accuracy Curves

The relationship between training and validation accuracy provides useful information about model performance.

Good Training Behavior

A healthy training process may look like:

```
Training Accuracy     ↑
Validation Accuracy   ↑
```

Both curves increase and remain relatively close to each other.

This generally indicates that the model is learning useful patterns and generalizing well.

Possible Overfitting

A possible sign of overfitting is:

```
Training Accuracy     ↑↑
Validation Accuracy   → or ↓
```

For example:

```
Training Accuracy   → 99.8%
Validation Accuracy → 97.0%
```

If the gap continues to increase, the model may be learning the training data too specifically.

Possible Underfitting

If both training and validation accuracy remain relatively low:

```
Training Accuracy     → Low
Validation Accuracy   → Low
```

the model may not be learning enough useful patterns from the data.

This situation is known as **underfitting**.

---

12. Example Interpretation

Suppose the graph shows:

```
Epoch 1
Training Accuracy   = 95%
Validation Accuracy = 96%

Epoch 5
Training Accuracy   = 98%
Validation Accuracy = 98%

Epoch 10
Training Accuracy   = 99%
Validation Accuracy = 98.5%
```

This indicates that:

- The model's training performance improves over time.
- Validation performance also improves.
- The difference between training and validation accuracy remains relatively small.
- The model is likely generalizing reasonably well.

The exact interpretation should always be based on the actual graph produced by the training process.

---

13. Relationship with Early Stopping

The accuracy plot is especially useful when combined with the Early Stopping callback.

The model was configured with:

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

Notice that Early Stopping monitors:

```
val_loss
```

while this graph displays:

```
accuracy
val_accuracy
```

Therefore, the graph helps visualize accuracy trends, while Early Stopping makes its stopping decision based on **validation loss**.

---

14. Data Used in the Plot

The graph obtains its values from:

```
history.history
```

The training process stores information such as:

```
history.history['loss']
history.history['accuracy']
history.history['val_loss']
history.history['val_accuracy']
```

For this particular graph, only these two are required:

```
history.history['accuracy']
history.history['val_accuracy']
```

---

15. Complete Code

```
# Plot Accuracy

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```

---

16. Summary

Code	Purpose
`plt.figure()`	Creates the figure
`history.history['accuracy']`	Gets training accuracy
`history.history['val_accuracy']`	Gets validation accuracy
`plt.plot()`	Draws the accuracy curves
`plt.title()`	Adds graph title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.legend()`	Identifies the curves
`plt.grid(True)`	Adds grid lines
`plt.show()`	Displays the graph

Conclusion

The **Training vs Validation Accuracy** graph is an important visualization for analyzing the learning behavior of the CNN.

By comparing the two curves, we can determine whether the model is:

- Learning effectively.
- Generalizing well.
- Potentially overfitting.
- Potentially underfitting.

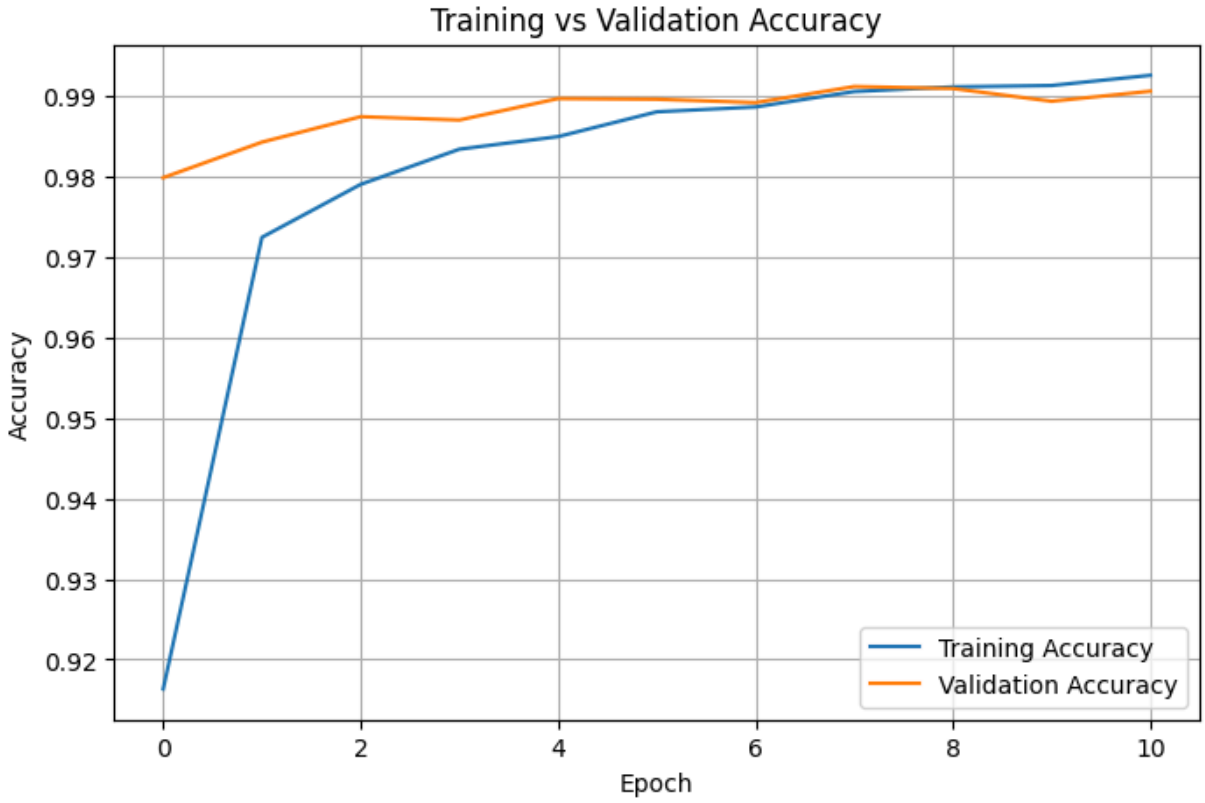
This visualization, together with the test accuracy and test loss, provides a clearer understanding of the overall performance of the trained MNIST CNN model.

```
# Plot Accuracy
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accur
plt.plot(history.history['val_accuracy'], label='Validation

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Plotting Training and Validation Loss

After training the CNN model, the `history` object contains the loss values recorded during each epoch.

Plotting the training and validation loss helps us understand how the model's error changes during training and whether the model is learning effectively.

Code

```
# Plot Loss

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)
```

```
plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```

---

1. Purpose of the Loss Plot

The loss plot compares two important values:

- **Training Loss**
- **Validation Loss**

The graph shows how the model's prediction error changes over the training epochs.

A good training process generally shows a decreasing loss:

```
Loss
 |\
 | \
 |  \
 |   \
 |    \_______
 |
 └──────────────────
       Epoch
```

Lower loss generally indicates that the model is making predictions that are closer to the correct labels.

---

2. Creating the Figure

```
plt.figure(figsize=(8, 5))
```

This creates a new figure for the loss graph.

The `figsize` parameter controls the size of the figure:

```
Width  = 8 inches
Height = 5 inches
```

This provides enough space to clearly display the two loss curves.

---

3. Plotting Training Loss

```
plt.plot(
    history.history['loss'],
    label='Training Loss'
)
```

The training loss values are stored in:

```
history.history['loss']
```

These values represent the loss calculated on the training data after each epoch.

For example:

```
Epoch 1 → 0.20
Epoch 2 → 0.12
Epoch 3 → 0.08
Epoch 4 → 0.06
```

If the training loss decreases over time, it generally indicates that the model is learning from the training data.

The label:

```
Training Loss
```

is used to identify this curve in the legend.

---

4. Plotting Validation Loss

```
plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)
```

The validation loss values are stored in:

```
history.history['val_loss']
```

These values represent the model's loss on the validation dataset after each epoch.

The validation data is the portion of the training data reserved through:

```
validation_split=0.2
```

The validation loss is especially important because it provides an indication of how well the model performs on data that was not used directly to update its weights.

---

5. Adding the Graph Title

```
plt.title("Training vs Validation Loss")
```

This adds a descriptive title to the graph.

The title indicates that the graph compares:

```
Training Loss
      vs
Validation Loss
```

---

6. X-Axis Label

```
plt.xlabel("Epoch")
```

The x-axis represents the number of training epochs.

For example:

```
1 → First epoch
2 → Second epoch
3 → Third epoch
...
```

If Early Stopping terminates training early, the graph may contain fewer points than the maximum number of epochs specified in `model.fit()`.

---

7. Y-Axis Label

```
plt.ylabel("Loss")
```

The y-axis represents the loss value.

Loss measures how different the model's predictions are from the correct labels.

Generally:

```
Lower Loss → Better Prediction
Higher Loss → Larger Prediction Error
```

The exact numerical range depends on the loss function and the model's predictions.

---

8. Adding the Legend

```
plt.legend()
```

The legend identifies the two curves:

```
Training Loss
Validation Loss
```

This makes it easier to compare the behavior of the model on training and validation data.

---

9. Adding Grid Lines

```
plt.grid(True)
```

This adds grid lines to the graph.

Grid lines make it easier to estimate loss values and observe changes between epochs.

---

10. Displaying the Graph

```
plt.show()
```

The `plt.show()` function displays the completed graph.

It should normally be placed at the end of the plotting code.

---

11. Understanding Training Loss

Training loss indicates how well the model fits the training data.

A typical learning pattern is:

```
Epoch 1 → High Training Loss
Epoch 2 → Lower Training Loss
Epoch 3 → Lower Training Loss
Epoch 4 → Lower Training Loss
...
```

A continuously decreasing training loss generally indicates that the model is learning patterns from the training dataset.

---

12. Understanding Validation Loss

Validation loss measures the model's performance on the validation data.

Ideally, validation loss should also decrease as training progresses:

```
Training Loss     ↓
Validation Loss   ↓
```

When both losses decrease and remain relatively close, the model is generally learning and generalizing well.

13. Detecting Overfitting

The loss plot is particularly useful for identifying **overfitting**.

A common overfitting pattern is:

```
Training Loss     ↓↓↓
Validation Loss   ↓ then ↑
```

For example:

```
Epoch 1 → Training Loss = 0.20, Validation Loss = 0.18
Epoch 2 → Training Loss = 0.10, Validation Loss = 0.09
Epoch 3 → Training Loss = 0.05, Validation Loss = 0.07
Epoch 4 → Training Loss = 0.03, Validation Loss = 0.10
Epoch 5 → Training Loss = 0.02, Validation Loss = 0.14
```

Here, training loss continues to decrease while validation loss starts increasing.

This can indicate that the model is becoming too specialized to the training data.

---

14. Detecting Good Generalization

A desirable pattern is:

```
Training Loss     ↓
Validation Loss   ↓
```

and the two curves remain reasonably close.

For example:

```
Epoch 1 → Training = 0.20, Validation = 0.21
Epoch 2 → Training = 0.12, Validation = 0.13
Epoch 3 → Training = 0.08, Validation = 0.09
Epoch 4 → Training = 0.06, Validation = 0.07
```

This suggests that the model is improving on both training and validation data.

---

15. Relationship with Early Stopping

The loss plot is directly related to the Early Stopping callback used earlier:

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)
```

The callback monitors:

```
val_loss
```

Therefore, if the validation loss stops improving for the specified patience period, training can be stopped.

For example:

```
Epoch 1 → val_loss = 0.10
Epoch 2 → val_loss = 0.07
Epoch 3 → val_loss = 0.05  ← Best
Epoch 4 → val_loss = 0.06
Epoch 5 → val_loss = 0.07
Epoch 6 → val_loss = 0.08
```

The model can stop training after the patience condition is reached and restore the weights associated with the best validation loss.

---

16. Difference Between Loss and Accuracy

Loss and accuracy measure different aspects of model performance.

Accuracy

Accuracy measures the proportion of predictions that are correct.

```
Accuracy ↑ → Generally better
```

Loss

Loss measures the degree of error in the model's predictions.

```
Loss ↓ → Generally better
```

Therefore, during successful training, we generally expect:

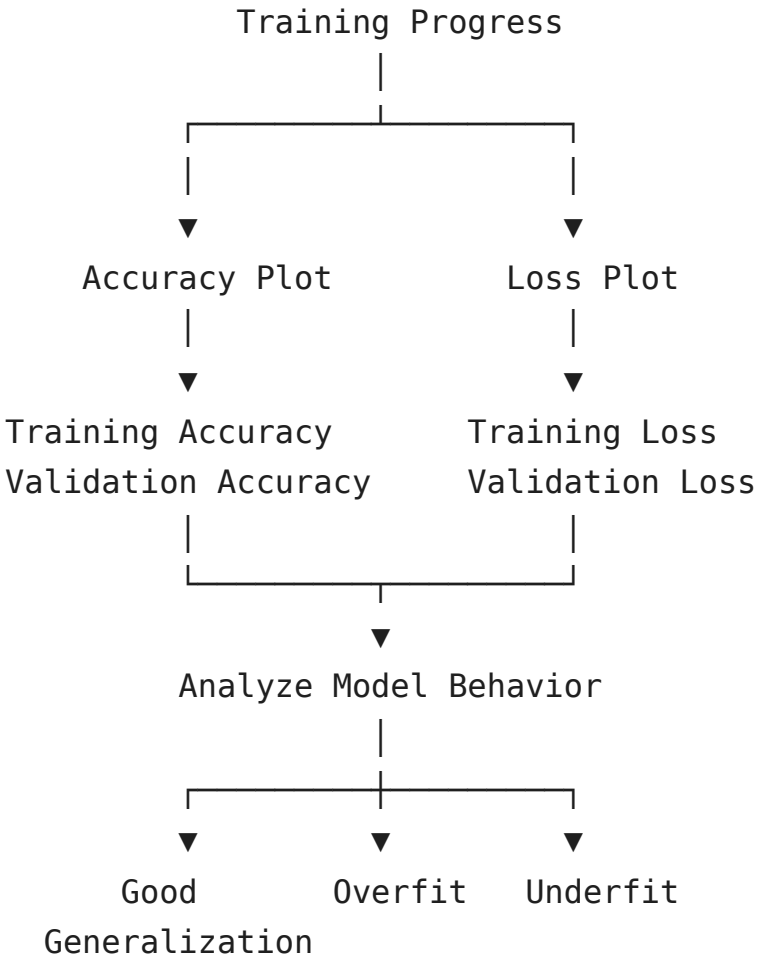
```
Accuracy     ↑
Loss         ↓
```

Both training and validation curves should be considered together.

---

17. Complete Training Analysis

The accuracy and loss plots can be used together:



---

18. Complete Code

```
# Plot Loss

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```

---

19. Summary

Code	Purpose
`plt.figure()`	Creates the graph
`history.history['loss']`	Retrieves training loss
`history.history['val_loss']`	Retrieves validation loss
`plt.plot()`	Draws the loss curves
`plt.title()`	Adds the graph title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.legend()`	Identifies the curves
`plt.grid(True)`	Adds grid lines
`plt.show()`	Displays the graph

20. Conclusion

The **Training vs Validation Loss** plot is an important tool for analyzing the learning behavior of the CNN.

A successful training process generally shows decreasing training and validation loss. If training loss continues to decrease while validation loss begins to increase, it may indicate **overfitting**.

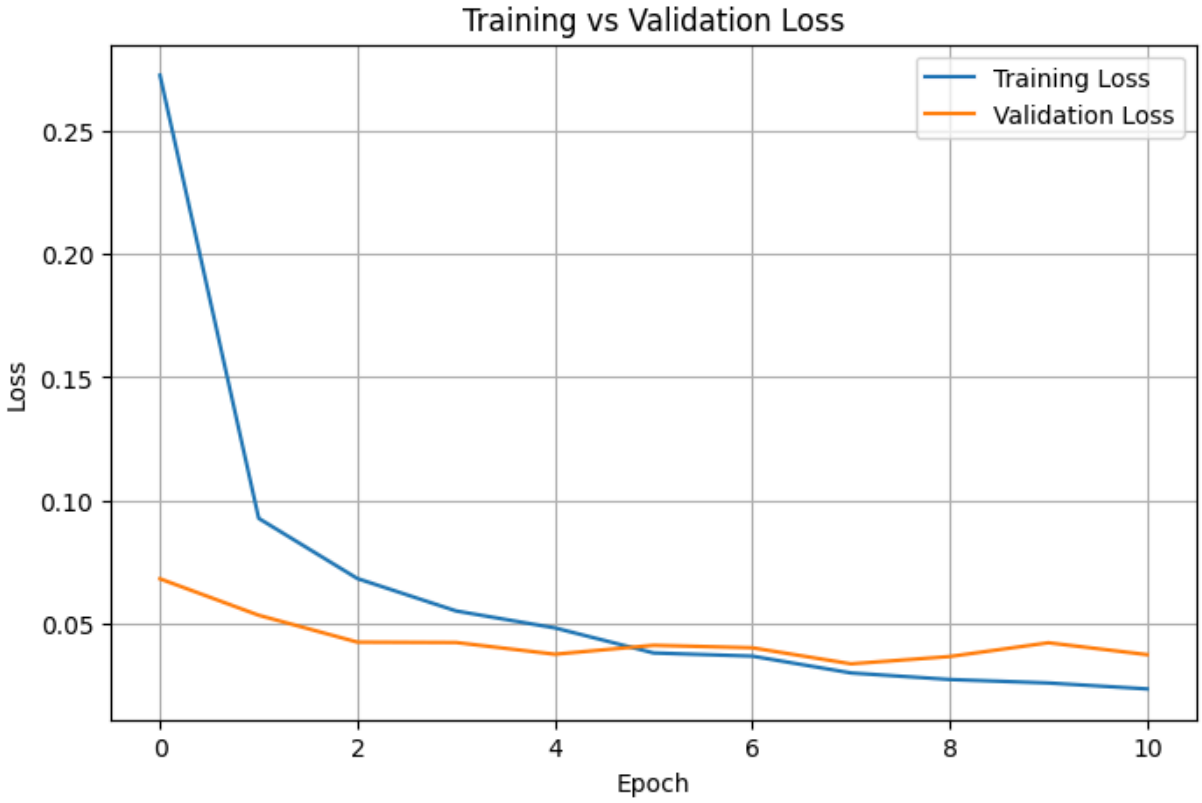
Together with the **Training vs Validation Accuracy** plot, the loss graph provides a clear visual understanding of how the CNN learns and generalizes during training.

```
# Plot Loss
plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Los

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Visualizing Random Test Predictions

After evaluating the CNN model, it is useful to visually inspect some of its predictions.

The following code randomly selects **20 test images**, uses the trained CNN to predict each digit, and displays the images in a **4 × 5 grid**.

The predicted digit is compared with the actual digit:

- **Green** → Correct prediction
- **Red** → Incorrect prediction

Code

```
random_indices = np.random.choice(
    len(X_test),
    20,
    replace=True
)
```

```
fig, axes = plt.subplots(4, 5, figsize=(12, 10))
axes = axes.ravel()

for i, idx in enumerate(random_indices):

    prediction = model.predict(
        X_test[idx:idx+1],
        verbose=0
    )

    predicted_digit = np.argmax(prediction)
    actual_digit = y_test[idx]

    axes[i].imshow(
        X_test[idx].reshape(28, 28),
        cmap='gray'
    )

    color = 'green' if predicted_digit == actual_digit else 'red'

    axes[i].set_title(
        f"Pred: {predicted_digit}\nActual: {actual_digit}",
        color=color,
        fontsize=10
    )

    axes[i].axis('off')

plt.tight_layout()
plt.show()
```

---

1. Purpose of This Visualization

The main purpose of this code is to **visually inspect the model's predictions**.

Instead of looking only at numerical metrics such as:

```
Test Accuracy
Test Loss
```

we can directly see individual test images and determine whether the CNN classified them correctly.

The visualization follows this process:

```
Test Dataset
     ↓
Randomly Select 20 Images
     ↓
CNN Prediction
     ↓
Compare Prediction with Actual Label
     ↓
Display Image
     ↓
Green = Correct
Red = Incorrect
```

---

2. Selecting Random Test Images

```
random_indices = np.random.choice(
    len(X_test),
    20,
    replace=True
)
```

This line randomly selects **20 indices** from the test dataset.

`len(X_test)`

```
len(X_test)
```

returns the total number of test images.

For MNIST:

```
len(X_test) = 10,000
```

Therefore, the indices are selected from the range of available test images.

---

3. Number of Images

```
20
```

The second argument specifies that **20 indices** should be selected.

Therefore, the visualization will contain 20 test images.

---

4. `replace=True`

```
replace=True
```

This allows an index to be selected more than once.

For example:

```
[102, 450, 102, 782, ...]
```

The same test image can therefore appear multiple times.

If you want to guarantee that all 20 selected images are different, you can use:

```
replace=False
```

For example:

```
random_indices = np.random.choice(
    len(X_test),
    20,
    replace=False
)
```

This is generally preferable when the goal is to inspect 20 unique examples.

---

5. Creating the Visualization Grid

```
fig, axes = plt.subplots(
    4,
    5,
    figsize=(12, 10)
)
```

This creates a figure containing a grid of subplots.

The grid consists of:

```
4 rows × 5 columns = 20 plots
```

Therefore, each selected test image has its own subplot.

The layout looks like:

```
┌──────┬──────┬──────┬──────┬──────┐
│  1   │  2   │  3   │  4   │  5   │
├──────┼──────┼──────┼──────┼──────┤
│  6   │  7   │  8   │  9   │ 10   │
├──────┼──────┼──────┼──────┼──────┤
│ 11   │ 12   │ 13   │ 14   │ 15   │
├──────┼──────┼──────┼──────┼──────┤
│ 16   │ 17   │ 18   │ 19   │ 20   │
└──────┴──────┴──────┴──────┴──────┘
```

---

6. Figure Size

```
figsize=(12, 10)
```

This determines the size of the overall figure.

Here:

- Width = 12 inches
- Height = 10 inches

A larger figure provides enough space for the 20 images and their labels.

7. Flattening the Axes Array

```
axes = axes.ravel()
```

When `plt.subplots(4, 5)` is used, the `axes` object is initially arranged as a 2D array:

```
4 × 5
```

The `ravel()` function converts it into a one-dimensional array:

```
[axis1, axis2, axis3, ..., axis20]
```

This makes it easier to access each subplot inside the loop.

For example:

```
axes[0]
axes[1]
axes[2]
...
axes[19]
```

---

8. Looping Through the Random Images

```
for i, idx in enumerate(random_indices):
```

This loop processes each randomly selected test image.

Two variables are used:

`i`

Represents the position of the image in the visualization.

```
0 → First subplot
1 → Second subplot
2 → Third subplot
```

```
...
19 → Last subplot
```

`idx`

Represents the actual index of the selected test image.

For example:

```
i   = 0
idx = 452
```

means that the first subplot displays the test image at index 452.

---

9. Making a Prediction

```
prediction = model.predict(
    X_test[idx:idx+1],
    verbose=0
)
```

The trained CNN is used to predict the selected test image.

The expression:

```
X_test[idx:idx+1]
```

selects exactly one image while preserving the batch dimension.

For example, if:

```
idx = 100
```

then:

```
X_test[100:101]
```

returns one image with a shape similar to:

```
(1, 28, 28, 1)
```

This is the expected batch format for the CNN.

---

10. `verbose=0`

```
verbose=0
```

This prevents `model.predict()` from displaying a progress bar for every individual image.

Without it, the output could become cluttered because the prediction function is called 20 times.

---

11. Finding the Predicted Digit

```
predicted_digit = np.argmax(prediction)
```

The CNN's final layer uses the `softmax` activation function and produces probabilities for the ten digit classes.

For example:

```
Digit 0 → 0.01
Digit 1 → 0.02
Digit 2 → 0.01
Digit 3 → 0.03
Digit 4 → 0.01
Digit 5 → 0.87
Digit 6 → 0.01
Digit 7 → 0.01
Digit 8 → 0.01
Digit 9 → 0.02
```

`np.argmax()` returns the index of the largest value.

Here:

```
0.87 → Digit 5
```

Therefore:

```
predicted_digit = 5
```

The model's prediction is **digit 5**.

---

12. Getting the Actual Digit

```
actual_digit = y_test[idx]
```

The actual digit is obtained from the test label corresponding to the selected image.

For example:

```
y_test[idx] = 5
```

means the actual digit is:

```
5
```

The model's prediction can then be compared with the actual label.

---

13. Displaying the Test Image

```
axes[i].imshow(
    X_test[idx].reshape(28, 28),
    cmap='gray'
)
```

The selected test image is displayed using `imshow()`.

The image was previously reshaped for the CNN into:

```
28 × 28 × 1
```

For visualization, the channel dimension is removed:

```
.reshape(28, 28)
```

This produces the original 2D grayscale image.

The parameter:

```
cmap='gray'
```

displays the image using a grayscale color map.

---

14. Comparing Prediction and Actual Label

```
color = 'green' if predicted_digit == actual_digit else 'red'
```

This line determines the title color based on whether the prediction is correct.

Correct Prediction

If:

```
predicted_digit == actual_digit
```

the color is:

```
green
```

Incorrect Prediction

If:

```
predicted_digit != actual_digit
```

the color is:

```
red
```

Therefore:

```
Green → Correct
Red   → Incorrect
```

---

15. Adding Prediction Information

```
axes[i].set_title(
    f"Pred: {predicted_digit}\nActual: {actual_digit}",
    color=color,
    fontsize=10
)
```

This adds a title above each image.

For example:

```
Pred: 7
Actual: 7
```

indicates a correct prediction.

An incorrect prediction might look like:

```
Pred: 3
Actual: 8
```

The title color indicates whether the prediction was correct.

---

16. New Line in the Title

The following expression:

```
"\n"
```

creates a line break.

Therefore:

```
f"Pred: {predicted_digit}\nActual: {actual_digit}"
```

produces:

```
Pred: 7
Actual: 7
```

instead of displaying both values on the same line.

17. Font Size

```
fontsize=10
```

This controls the size of the prediction text.

A font size of 10 is suitable for displaying labels above each image without taking too much space.

---

18. Removing Axis Information

```
axes[i].axis('off')
```

This removes the x-axis and y-axis markings from the individual image.

Since the purpose of the visualization is to inspect handwritten digits, numerical axis values are unnecessary.

The result is a cleaner visualization.

---

19. Adjusting the Layout

```
plt.tight_layout()
```

`tight_layout()` automatically adjusts the spacing between the subplots.

This helps prevent:

- Titles from overlapping.
- Images from being too close together.
- Labels from being cut off.

It is especially useful when displaying multiple images in a grid.

---

20. Displaying the Figure

```
plt.show()
```

Finally, `plt.show()` displays the complete 4 × 5 image grid.

The resulting visualization contains:

```
20 Test Images
      ↓
CNN Predictions
      ↓
Prediction vs Actual Label
      ↓
Green = Correct
Red = Incorrect
```

---

21. Example Interpretation

Suppose one subplot displays:

```
Pred: 6
Actual: 6
```

in green.

This means:

```
Model Prediction = 6
Actual Label     = 6
Result           = Correct
```

If another subplot displays:

```
Pred: 4
Actual: 9
```

in red, it means:

```
Model Prediction = 4
Actual Label     = 9
Result           = Incorrect
```

This allows us to visually inspect both successful and unsuccessful predictions.

---

22. Why This Visualization Is Useful

Numerical accuracy alone does not show **which images the model gets wrong**.

This visualization helps us:

1. Inspect individual predictions.
2. Identify correct classifications.
3. Identify misclassified digits.
4. Understand difficult handwritten patterns.
5. Visually verify model performance.
6. Build intuition about the CNN's classification behavior.

For example, some handwritten digits may be difficult to distinguish:

```
3 ↔ 8
4 ↔ 9
5 ↔ 6
1 ↔ 7
```

Visualizing incorrect predictions can help identify such cases.

---

23. Important Note About `replace=True`

The current code uses:

```
replace=True
```

This means the same image can potentially appear more than once.

If the objective is to display **20 unique random test images**, it is better to use:

```
random_indices = np.random.choice(
    len(X_test),
    20,
    replace=False
)
```

This ensures that no test image is selected more than once.

---

24. Complete Visualization Code

```
# Select 20 random test images
random_indices = np.random.choice(
    len(X_test),
    20,
    replace=False
)

# Create a 4 × 5 grid
fig, axes = plt.subplots(
    4,
    5,
    figsize=(12, 10)
)

# Convert axes into a 1D array
axes = axes.ravel()

# Display predictions
for i, idx in enumerate(random_indices):

    # Make prediction
    prediction = model.predict(
        X_test[idx:idx+1],
        verbose=0
    )

    # Get predicted and actual digits
    predicted_digit = np.argmax(prediction)
    actual_digit = y_test[idx]

    # Display image
    axes[i].imshow(
        X_test[idx].reshape(28, 28),
        cmap='gray'
    )
```

```
    # Set title color
    color = (
        'green'
        if predicted_digit == actual_digit
        else 'red'
    )

    # Display prediction and actual label
    axes[i].set_title(
        f"Pred: {predicted_digit}\nActual: {actual_digit}",
        color=color,
        fontsize=10
    )

    # Hide axes
    axes[i].axis('off')

# Adjust layout
plt.tight_layout()

# Display figure
plt.show()
```

---

25. Summary

Code	Purpose
`np.random.choice()`	Selects random test images
`plt.subplots(4, 5)`	Creates 20-image grid
`axes.ravel()`	Converts subplot array to 1D
`model.predict()`	Predicts the digit
`np.argmax()`	Finds the predicted class
`y_test[idx]`	Gets the actual label
`imshow()`	Displays the test image
`green`	Correct prediction
`red`	Incorrect prediction
`axis('off')`	Hides axis markings

Code	Purpose
`tight_layout()`	Improves subplot spacing
`plt.show()`	Displays the final visualization

Conclusion

This visualization provides an intuitive way to evaluate the CNN's predictions on individual MNIST test images.

```
1  import numpy as np
```